

1. College Management — Aggregation

Create Student and Address classes.

- Address should contain city, state, and pincode.
 - Student should contain name and roll number.
 - A Student object should contain an Address object.
 - Display complete student information.
 - Demonstrate the HAS-A relationship.
-

2. Car & Engine — Aggregation

Create Engine and Car classes.

- Engine should have engine_no and cc.
 - Car should have model and price.
 - A Car should contain an Engine object.
 - Create a method to display car and engine details.
 - Start the engine through the Car object.
-

3. Bank Account — Encapsulation

Create a BankAccount class.

- Create private variable __balance.
 - Create deposit() and withdraw() methods.
 - Create get_balance() getter.
 - Don't allow withdrawal greater than balance.
 - Don't allow negative deposit.
 - Keep balance inaccessible directly from outside.
-

4. Student Marks — Encapsulation + Getter/Setter

Create a Student class.

- Private variable `__marks`.
 - Create getter and setter.
 - Setter should allow marks only between 0 and 100.
 - Create `display()` method.
 - Try changing marks with both valid and invalid values.
-

5. Employee — Instance & Static Variables

Create an Employee class.

- Instance variables: name, salary, department.
 - Class/static variable: company = "ABC Ltd".
 - Create 3 employee objects.
 - Display all employee information.
 - Change the company name using the class.
 - Observe the effect on all objects.
-

6. Student Management — Instance & Static Methods

Create a Student class.

- Instance variables: name, marks.
 - Class variable: school = "ABC School".
 - Instance method `display()` to show student details.
 - Static method `school_rules()` to display common school rules.
 - Call the instance method using an object.
 - Call the static method using the class.
-

7. Employee Salary — Class Variable + Instance Variable

Create an Employee class.

- company should be a class variable.
 - name and salary should be instance variables.
 - Create 3 employees.
 - Give each employee a different salary.
 - Change the company name.
 - Display all employees again.
-

8. Vehicle — Method Overriding

Create a parent class Vehicle.

- Create a method start().
 - Create child classes Car, Bike, and Bus.
 - Override start() in each child.
 - Create objects of all three classes.
 - Call start() and observe different outputs.
-

9. Payment System — Method Overriding + Polymorphism

Create a parent class Payment.

Create child classes:

Payment

└─ UPI

└─ CreditCard

└─ NetBanking

- Create pay() method in parent.
- Override pay() in each child.
- Create a function make_payment(payment).

- Pass different payment objects to it.
 - Observe different behavior.
-

10. Calculator — Method Overloading

Create a Calculator class.

Create an add() method that should support:

add(10, 20)

add(10, 20, 30)

add(10, 20, 30, 40)

- Use *args or default arguments.
 - Do not create multiple add() methods.
 - Similarly implement multiply() using *args.
-

11. Library Management — Aggregation + Encapsulation

Create:

Book

Library

- Book should contain title, author, and price.
 - Make price private.
 - Create getter/setter for price.
 - Library should contain multiple Book objects.
 - Add books to the library.
 - Display all books.
 - Search for a book by title.
-

12. E-Commerce Product — Encapsulation + Static Method

Create a Product class.

- Private variable `__price`.
 - Instance variables: name, quantity.
 - Getter/setter for price.
 - Instance method `total_price()`.
 - Static method `calculate_gst(price)` that calculates GST.
 - Display the final product price including GST.
-

13. Hospital Management — Aggregation + Inheritance

Create:

Person

└─ Doctor

└─ Patient

and an Address class.

- Doctor and Patient should inherit from Person.
- Both should contain an Address object.
- Add different attributes for Doctor and Patient.
- Override `display()` in both child classes.
- Use `super()` where required.

This combines inheritance + aggregation + overriding.
